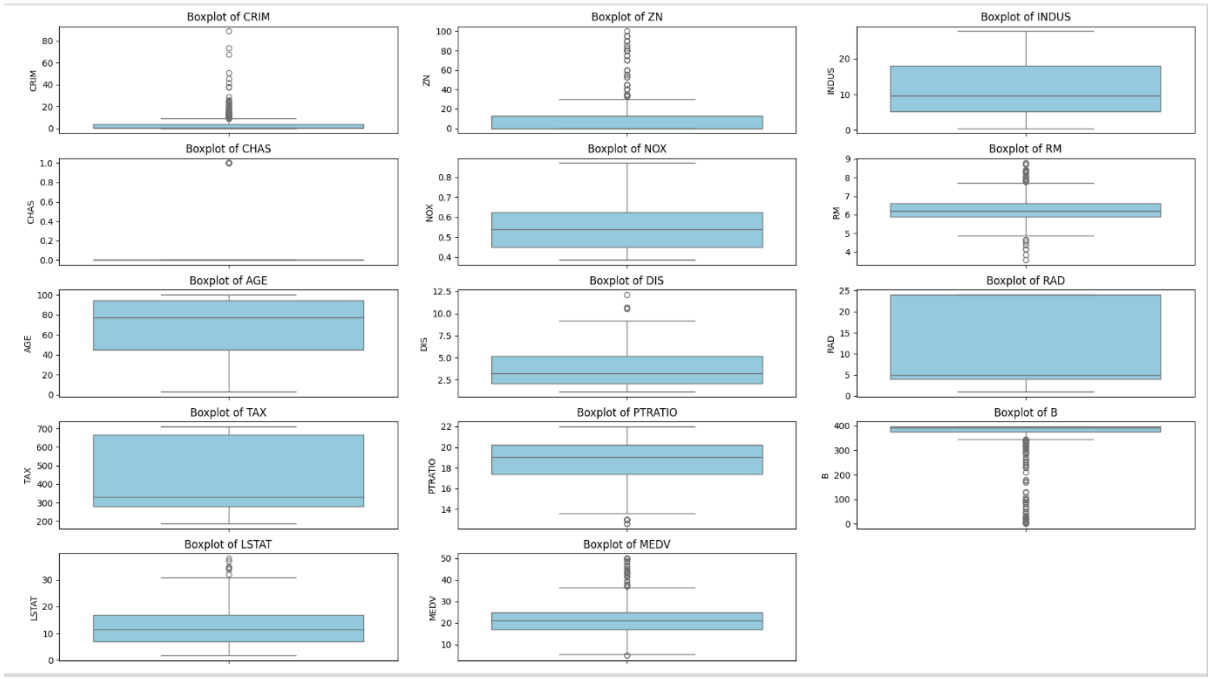
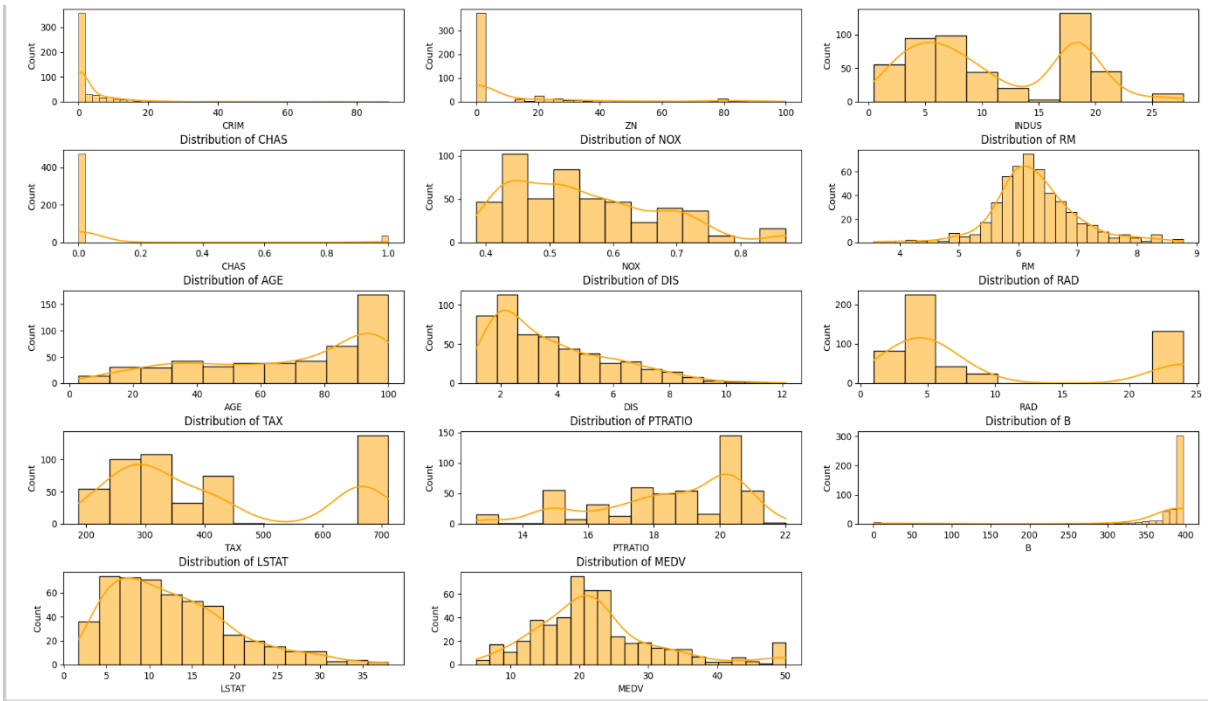
```
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("Dataset_Day5.csv")

# Check for Missing Values
print("Missing Values:")
print(df.isnull().sum())
print("\n")

# Descriptive Statistics
print("Descriptive Statistics:")
print(df.describe().T)
print("\n")

# Detect Outliers using Boxplots
print("Visualizing Outliers (Boxplots):")
plt.figure(figsize=(15, 12))
for i, column in enumerate(df.select_dtypes(include='number').columns):
    plt.subplot(5, 3, i+1)
    sns.boxplot(y=df[column], color='skyblue')
    plt.title(f'Boxplot of {column}')
plt.tight_layout()
plt.show()

# Histograms for Visualizing Distributions
print("Visualizing Data Distributions (Histograms):")
plt.figure(figsize=(15, 12))
for i, column in enumerate(df.select_dtypes(include='number').columns):
    plt.subplot(5, 3, i+1)
    sns.histplot(df[column], kde=True, color='orange')
    plt.title(f'Distribution of {column}')
```

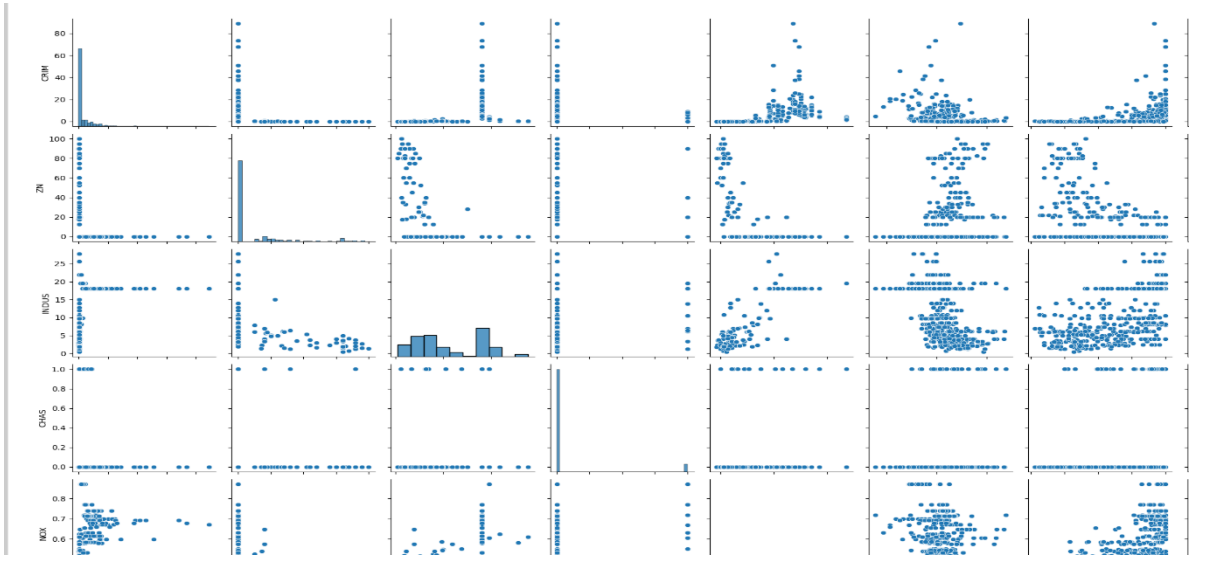
```
    sns.boxplot(y=df[column], color='skyblue')
    plt.title(f'Boxplot of {column}')
plt.tight_layout()
plt.show()

# Histograms for Visualizing Distributions
print("Visualizing Data Distributions (Histograms):")
plt.figure(figsize=(15, 12))
for i, column in enumerate(df.select_dtypes(include='number').columns):
    plt.subplot(5, 3, i+1)
    sns.histplot(df[column], kde=True, color='orange')
    plt.title(f'Distribution of {column}')
plt.tight_layout()
plt.show()

print("\nPairplot for continuous variables:")
sns.pairplot(df.select_dtypes(include=['float64', 'int64']))
plt.show()
```

```
Descriptive Statistics:
         count        mean         std  ...         50%         75%         max
CRIM     506.0    3.613524    8.601545  ...     0.25651    3.677083     88.9762
ZN       506.0   11.363636   23.322453  ...     0.00000   12.500000    100.0000
INDUS    506.0   11.136779    6.860353  ...     9.69000   18.100000     27.7400
CHAS     506.0    0.069170    0.253994  ...     0.00000    0.000000      1.0000
NOX      506.0    0.554695    0.115878  ...     0.53800    0.624000      0.8710
RM       506.0    6.284634    0.702617  ...     6.20850    6.623500      8.7800
AGE      506.0   68.574901   28.148861  ...    77.50000   94.075000    100.0000
DIS      506.0    3.795043    2.105710  ...     3.20745    5.188425     12.1265
RAD      506.0    9.549407    8.707259  ...     5.00000   24.000000     24.0000
TAX      506.0  408.237154  168.537116  ...   330.00000  666.000000    711.0000
PTRATIO  506.0   18.455534    2.164946  ...    19.05000   20.200000     22.0000
B        506.0  356.674032   91.294864  ...   391.44000  396.225000    396.9000
LSTAT    506.0   12.653063    7.141062  ...    11.36000   16.955000     37.9700
MEDV     506.0   22.532806    9.197104  ...    21.20000   25.000000     50.0000

[14 rows x 8 columns]
```

```
CRIM       0
ZN         0
INDUS      0
CHAS       0
NOX        0
RM         0
AGE        0
DIS        0
RAD        0
TAX        0
PTRATIO    0
B          0
LSTAT      0
MEDV       0
dtype: int64
```


Boxplot of CRIM
Boxplot of ZN
Boxplot of INDUS
Boxplot of CHAS
Boxplot of NOX
Boxplot of RM
Boxplot of AGE
Boxplot of DIS
Boxplot of RAD
Boxplot of TAX
Boxplot of PTRATIO
Boxplot of B
Boxplot of LSTAT
Boxplot of MEDV


Distribution of CHAS
Distribution of NOX
Distribution of RM
Distribution of AGE
Distribution of DIS
Distribution of RAD
Distribution of TAX
Distribution of PTRATIO
Distribution of B
Distribution of LSTAT
Distribution of MEDV



```python
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
import pandas as pd
df = pd.read_csv("Dataset_Day5.csv")

# Features and target
X_dis = df [['DIS']]  # distance to employment centers
y = df ['MEDV']

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_dis, y, test_size=0.2, random_state=100)

# Train the model
model = LinearRegression()
model.fit(X_train, y_train)
# Show regression equation
print("Regression Equation:")
print(f"MEDV = {model.intercept_:.2f} + {model.coef_[0]:.2f} * DIS")
# R² Score
y_pred = model.predict(X_test)
print(f"R² Score: {r2_score(y_test, y_pred):.4f}")
# Predict MEDV for DIS = 15
predicted_value = model.predict([[15]])[0]
print(f"Predicted MEDV when DIS = 15: {predicted_value:.2f}")
```

```
Regression Equation:
MEDV = 18.38 + 1.12 * DIS
R² Score: 0.0541
Predicted MEDV when DIS = 15: 35.16
```

```python
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
import pandas as pd

df = pd.read_csv("Dataset_Day5.csv")
df = df.dropna()

X = df[['RM']]
y = df['MEDV']

#Training
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=100)

# Train the model
model = LinearRegression()
model.fit(X_train, y_train)

# Print the equation
print("Regression Equation:")
print(f"MEDV = {model.intercept_:.2f} + {model.coef_[0]:.2f} * RM")

#  R² score
y_pred = model.predict(X_test)
print(f"R² Score: {r2_score(y_test, y_pred):.4f}")

#  Predict MEDV for RM = 7
predicted_value = model.predict([[7]])[0]
print(f"Predicted MEDV when RM = 7: {predicted_value:.2f}")
```

```
Regression Equation:
MEDV = -33.11 + 8.88 * RM
R² Score: 0.4885
```

```
Predicted MEDV when RM = 7: 29.05
```